# Chapter 16
# Remedies for Severe Class Imbalance

When modeling discrete classes, the relative frequencies of the classes can have a significant impact on the effectiveness of the model. An imbalance occurs when one or more classes have very low proportions in the training data as compared to the other classes. Imbalance can be present in any data set or application, and hence, the practitioner should be aware of the implications of modeling this type of data.

Here are a few practical settings where class imbalance often occurs:

- Online advertising: An advertisement is presented to a viewer which creates an *impression*. The *click through rate* is the number of times an ad was clicked on divided by the total number of impressions and tends to be very low (Richardson et al. 2007 cite a rate less than 2.4%).
- Pharmaceutical research: High-throughput screening is an experimental technique where large numbers of molecules (10000s) are rapidly evaluated for biological activity. Usually only a few molecules show high activity; therefore, the frequency of interesting compounds is low.
- Insurance claims: Artis et al. (2002) investigated auto insurance damage claims in Spain between the years of 1993 and 1996. Of claims undergoing auditing, the rate of fraud was estimated to be approximately 22 %.

This chapter discusses the impact of class imbalance on performances measures, methodologies for post-processing model predictions, and predictive models that can mitigate the issue during training. However, before proceeding, we will describe another case study which we will use to illustrate the various approaches to addressing class imbalance.

## 16.1 Case Study: Predicting Caravan Policy Ownership

A data set generated by the computational intelligence and learning (CoIL) research network is used to illustrate methods for combatting class imbalances. The 2000 CoIL Challenge was to predict whether customers would purchase

M. Kuhn and K. Johnson, *Applied Predictive Modeling*,
DOI 10.1007/978-1-4614-6849-3_16,
© Springer Science+Business Media New York 2013

caravan insurance[1]; Van Der Putten and Van Someren (2004) discuss these data, the challenge, and some of the solutions to the problem. Our discussion of these data is limited to illustrating the effect of class imbalances.

The outcome, whether the costumer purchased caravan insurance, is highly unbalanced with only 6 % of customers having purchased policies. The predictors in the data set consisted of:

- Customer subtype designation, such as "Traditional Families" or "Affluent Young Families." There were 39 unique values, although many of the subtypes comprise less than 5 % of the customers.
- Demographic factors, such as religion, education level, social class, income, and 38 others. The values of the predictors were derived from data at the zip code level, so customers residing in the same zip code will have the same values for these attributes.[2]
- Product ownership information, such as the number of (or the contribution to) policies of various types.

In all there were 85 predictors. Many of the categorical predictors had 10 or more levels and the count-based predictors tended to be fairly sparse (i.e., few nonzero values).

To demonstrate different methodologies with these data, stratified random sampling (where the strata was the response variable) was used to create three different data sets:

- A training set of customers ($n = 6877$) that will be used to estimate model parameters, tuning models, etc.
- A small *evaluation* set of customers ($n = 983$) that will be used for developing post-processing techniques, such as alternative probability cutoffs
- A customer test set ($n = 1962$) that is solely used for final evaluations of the models

The rate of customers with caravan policies in each of these data sets was roughly the same across all three data sets (6 %, 5.9 %, and 6 %, respectively).

## 16.2 The Effect of Class Imbalance

To begin, three predictive models were used to model the data: random forest, a flexible discriminant analysis model (with MARS hinge functions), and logistic regression. To tune the models, 10-fold cross-validation was used; each holdout sample contained roughly 687 customers, which should provide

[1] We would like to thank Peter van der Putten for permission to use these data.

[2] Giving all customers within the same zip code the same values for the predictors implies that there is unseen noise within the predictors. The implications of predictor set noise are discussed in Sect. 20.3.

Table 16.1: Results for three predictive models using the evaluation set

| Model | Accuracy | Kappa | Sensitivity | Specificity | ROC AUC |
|---|---|---|---|---|---|
| Random forest | 93.5 | 0.091 | 6.78 | 99.0 | 0.757 |
| FDA (MARS) | 93.8 | 0.024 | 1.69 | 99.7 | 0.754 |
| Logistic regression | 93.9 | 0.027 | 1.69 | 99.8 | 0.727 |

reasonable estimates of uncertainty. To choose the optimal model, the area under the receiver operating characteristic (ROC) curve was optimized.[3]

The random forest model used 1500 trees in the forest and was tuned over 5 values of the $m_{try}$ parameter (Sect. 8.5); the final model had an optimal $m_{try}$ value of 126. The FDA model used first-degree features and was tuned over 25 values for the number of retained terms. The resampling process determined that 13 model terms was appropriate. Logistic regression utilized a simple additive model (i.e., no interactions or nonlinear terms) with a reduced predictor set (many near-zero variance predictors were removed so that the model resulted in a stable solution).

A number of different performance metrics were estimated including: overall accuracy, Kappa, area under the ROC curve, sensitivity, and specificity (where a purchased policy was defined as the "event" of interest). All models predicted the samples in the evaluation data set and yielded very similar results, as shown in Table 16.1. In each model, any patterns that were useful for predicting the outcome were overwhelmed by the large percentage of customers with no caravan insurance. In fact, none of the models predicted more than 13 customers on the evaluation set as having insurance, despite 59 customers with insurance in the evaluation set. The implication of these results is that the models achieve good specificity (since almost every customer is predicted no insurance) but have poor sensitivity.

The imbalance also had a severe effect on the predicted class probabilities. In the random forest model, for example, 82 % of the customers have a predicted probability of having insurance of 10 % or less. This highly left-skewed predicted probability distribution also occurs for the other two models.

Figure 16.1 shows the lift plots and ROC curves for the evaluation set. The lift curves are very similar and can be used to determine how many individuals would need to be contacted in order to capture a specific percent of those who might purchase the policy. For example, to find about 60 % of those with policies using the random forest model, around 30 % of the population would need to be sampled. The ROC curves show considerable overlap and does not differentiate the models.

[3] The results of the tuning process are summarized here. The full details of the process can be found in the `Chapters` directory of the AppliedPredictiveModeling package and in the Computing section of this chapter.

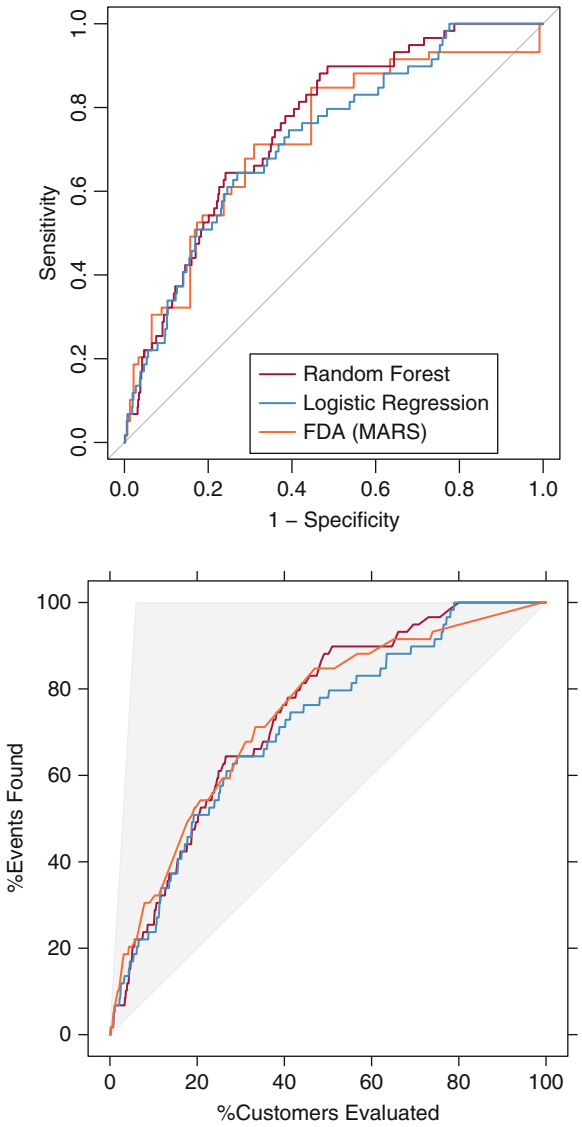



Fig. 16.1: *Top*: Evaluation set ROC curves for each of the three baseline models. *Bottom*: The corresponding lift plots

In this figure, there is a strong similarity between the lift and ROC curves. When classes are more balanced, lift plots and ROC curves are not this similar; rather, the similarity we see in Fig. 16.1 is due to class imbalance. In general, this is not the case but is an artifact of the severe class imbalance. When the classes are more balanced, the curves are unlikely to have similar

patterns. For example, the maximum area on the lift curve is bounded by the event rate in the data (as shown with the grey background area in the loft plot) while the ROC curve has no such limitation.

The remaining sections discuss other strategies for overcoming class imbalances. The next section discusses how model tuning can be used to increase the sensitivity of the minority class while Sect. 16.4 demonstrates how alternative probability cutoffs can be derived from the data to improve the error rate on the minority class. This amounts to post-processing the model predictions to redefine the class predictions. Modifications of case weights and prior probabilities are also discussed. Another section examines how the training data can be altered to mitigate the class imbalance prior to model training. The last approach described in Sect. 16.6 demonstrates how, for some models, the model training process can be altered to emphasize the accuracy of the model for the less frequent class(es). In this case, the estimated model parameters are being modified instead of post-processing the model output.

## 16.3 Model Tuning

The simplest approach to counteracting the negative effects of class imbalance is to tune the model to maximize the accuracy of the minority class(es). For insurance prediction, tuning the model to maximize the sensitivity may help desensitize the training process to the high percentage of data without caravan policies in the training set. The random forest model that was tuned for these data did not show a meaningful trend in sensitivity across the tuning parameter. The FDA model did show a trend; as the number of model terms was increased, there was a rise in sensitivity from effectively 0 % for very simple models to 5.4 % when 16 terms were retained. This minor improvement in sensitivity comes at virtually no cost to specificity. Given that the increase in sensitivity is not high enough to be considered acceptable, this approach to solving the problem is not effective for this particular data set. The use of model tuning for this purpose is revisited in Sect. 16.8.

## 16.4 Alternate Cutoffs

When there are two possible outcome categories, another method for increasing the prediction accuracy of the minority class samples is to determine alternative cutoffs for the predicted probabilities which effectively changes the definition of a predicted event. The most straightforward approach is to use the ROC curve since it calculates the sensitivity and specificity across a continuum of cutoffs. Using this curve, an appropriate balance between sensitivity and specificity can be determined.

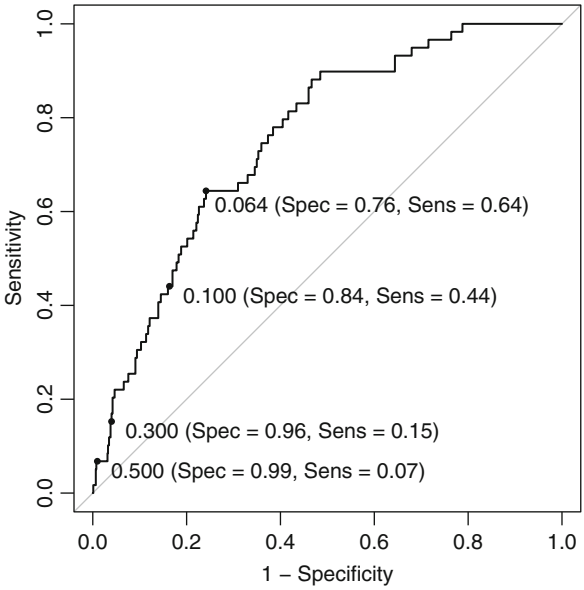



Fig. 16.2: The random forest ROC curve for predicting the classes using the evaluation set. The number on the left represents the probability cutoff, and the numbers in the parentheses are the specificity and sensitivity, respectively. Several possible probability cutoffs are used, including the threshold geometrically closest to the perfect model (0.064)

Figure 16.2 shows the ROC curve for the random forest model based on the evaluation set. Several cutoffs are shown on the curve and it is apparent that decreasing the cutoff for the probability of responding increases the sensitivity (at the expense of the specificity). There may be situations where the sensitivity/specificity trade-off can be accomplished without severely compromising the accuracy of the majority class (which, of course, depends on the context of the problem).

Several techniques exist for determining a new cutoff. First, if there is a particular target that must be met for the sensitivity or specificity, this point can be found on the ROC curve and the corresponding cutoff can be determined. Another approach is to find the point on the ROC curve that is closest (i.e., the shortest distance) to the perfect model (with 100 % sensitivity and 100 % specificity), which is associated with the upper left corner of the plot. In Fig. 16.2, a cutoff value of 0.064 would be the closest to the perfect model. Another approach for determining the cutoff uses Youden's $J$ index (see Sect. 11.2), which measures the proportion of correctly predicted samples for both the event and nonevent groups. This index can be computed for each cutoff that is used to create the ROC curve. The cutoff associated with the largest value of the Youden index may also show superior performance relative

Table 16.2: Confusion matrices for the test set for random forests using the default and alternate cutoffs

| | 0.50 Cutoff | | 0.064 Cutoff | |
|---|---|---|---|---|
| | Insurance | no insurance | Insurance | no insurance |
| Insurance | 11 | 19 | 71 | 441 |
| Noinsurance | 105 | 1827 | 45 | 1,405 |

to the default 50 % value. For the random forest ROC curve, the cutoff that maximizes the Youden index (0.021) is similar to the point closest to the optimal model.

Using the evaluation set, the predicted sensitivity for the new cutoff of 0.064 is 64.4 %, which is a significant improvement over the value generated by the default cutoff. The consequence of the new cutoff is that the specificity is estimated to drop from 99 % to 75.9 %. This may or may not be acceptable based on the context of how the model will be used.

In our analysis, the alternate cutoff for the model was not derived from the training or test sets. It is important, especially for small samples sizes, to use an independent data set to derive the cutoff. If the training set predictions are used, there is likely a large optimistic bias in the class probabilities that will lead to inaccurate assessments of the sensitivity and specificity. If the test set is used, it is no longer an unbiased source to judge model performance. For example, Ewald (2006) found via simulation that *post hoc* derivation of cutoffs can exaggerate test set performance.

Table 16.2 contains confusion matrices derived from the test set for the default and alternate cutoffs. Predictions from the 50 % cutoff echo the same performances values shown by cross-validation (a sensitivity of 9.5 % and a specificity of 99 %). For the new cutoff, the test set sensitivity was found to be 61.2 % while the specificity was calculated to be 76.1 %. This estimated difference in sensitivity between the two data sets (64.4 % and 61.2 %) is likely due to the high uncertainty in the metric (which itself is a result of the low frequency of events). These two values may be equivalent given the experimental noise. To reiterate, this cutoff resulted from being as close as possible to the optimal model. This may not be applicable in the context of other problems in which case alternative trade-offs can be made.

It is worth noting that the core of the model has not changed. The same model parameters are being used. Changing the cutoff to increase the sensitivity does not increase the overall predictive effectiveness of the model. The main impact that an alternative cutoff has is to make trade-offs between particular types of errors. For example, in a confusion matrix, alternate cutoffs can only move samples up and down rows of the matrix (as opposed to

moving them from the off-diagonals to the diagonals). In other words, using an alternative cutoff does not induce further separation between the classes.

Figure 16.2 and Table 16.2 demonstrate why, for many classification problems, comparing models on the basis of the default sensitivity and specificity may be misleading. Since a better cutoff may be possible, an analysis of the ROC curve can lead to improvements in these metrics. As shown in Fig. 16.2, a class imbalance can further exacerbate this issue. Consequently, performance metrics that are independent of probability cutoffs (such as the area under the ROC curve) are likely to produce more meaningful contrasts between models. However, some predictive models only produce discrete class predictions.

## 16.5 Adjusting Prior Probabilities

Some models use prior probabilities, such as naïve Bayes and discriminant analysis classifiers. Unless specified manually, these models typically derive the value of the priors from the training data. Weiss and Provost (2001a) suggest that priors that reflect the natural class imbalance will materially bias predictions to the majority class. Using more balanced priors or a balanced training set may help deal with a class imbalance.

For the insurance data, the priors are 6 % and 94 % for the insured and uninsured, respectively. The predicted probability of having insurance is extremely left-skewed for all three models and adjusting the priors can shift the probability distribution away from small values. For example, new priors of 60 % for the insured and 40 % for the uninsured in the FDA model increase the probability of having insurance significantly. With the default cutoff, predictions from the new model have a sensitivity of 71.2 % and a specificity of 66.9 % on the test set. However, the new class probabilities did not change the rankings of the customers in the test set and the model has the same area under the ROC curve as the previous FDA model. Like the previous tactics for an alternative cutoff, this strategy did not change the model but allows for different trade-offs between sensitivity and specificity.

## 16.6 Unequal Case Weights

Many of the predictive models for classification have the ability to use *case weights* where each individual data point can be given more emphasis in the model training phase. For example, previously discussed boosting approaches to classification and regression trees are able to create a sequence of models, each of which apply different case weights at each iteration.

One approach to rebalancing the training set would be to increase the weights for the samples in the minority classes (Ting 2002). For many models, this can be interpreted as having identical duplicate data points with the exact same predictor values. Logistic regression, for example, can utilize case weights in this way. This procedure for dealing with a class imbalance is related to the sampling methods discussed in next section.

## 16.7 Sampling Methods

When there is *a priori* knowledge of a class imbalance, one straightforward method to reduce its impact on model training is to select a training set sample to have roughly equal event rates during the initial data collection (see, e.g., Artis et al. 2002). Basically, instead of having the model deal with the imbalance, we can attempt to balance the class frequencies. Taking this approach eliminates the fundamental imbalance issue that plagues model training. However, if the training set is sampled to be balanced, the test set should be sampled to be more consistent with the state of nature and should reflect the imbalance so that honest estimates of future performance can be computed.

If an *a priori* sampling approach is not possible, then there are *post hoc* sampling approaches that can help attenuate the effects of the imbalance during model training. Two general *post hoc* approaches are *down-sampling* and *up-sampling* the data. Up-sampling is any technique that simulates or imputes additional data points to improve balance across classes, while down-sampling refers to any technique that reduces the number of samples to improve the balance across classes.

Ling and Li (1998) provide one approach to up-sampling in which cases from the minority classes are sampled with replacement until each class has approximately the same number. For the insurance data, the training set contained 6466 non-policy and 411 insured customers. If we keep the original minority class data, adding 6055 random samples (with replacement) would bring the minority class equal to the majority. In doing this, some minority class samples may show up in the training set with a fairly high frequency while each sample in the majority class has a single realization in the data. This is very similar to the case weight approach shown in an earlier section, with varying weights per case.

Down-sampling selects data points from the majority class so that the majority class is roughly the same size as the minority class(es). There are several approaches to down-sampling. First, a basic approach is to randomly sample the majority classes so that all classes have approximately the same size. Another approach would be to take a bootstrap sample across all cases such that the classes are balanced in the bootstrap set. The advantage of this approach is that the bootstrap selection can be run many times so that the estimate

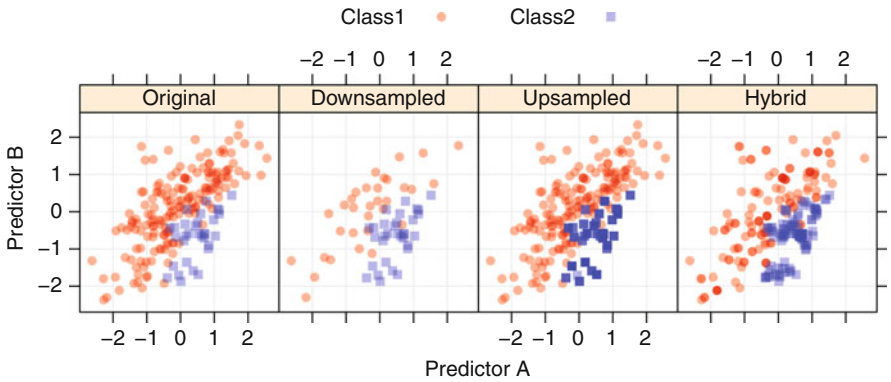



Fig. 16.3: *From left to right*: The original simulated data set and realizations of a down-sampled version, an up-sampled version, and sampling using SMOTE where the cases are sampled and/or imputed

of variation can be obtained about the down-sampling. One implementation of random forests can inherently down-sample by controlling the bootstrap sampling process within a *stratification variable*. If class is used as the stratification variable, then bootstrap samples will be created that are roughly the same size per class. These *internally down-sampled* versions of the training set are then used to construct trees in the ensemble.

The synthetic minority over-sampling technique (SMOTE), described by Chawla et al. (2002), is a data sampling procedure that uses both up-sampling and down-sampling, depending on the class, and has three operational parameters: the amount of up-sampling, the amount of down-sampling, and the number of neighbors that are used to impute new cases. To up-sample for the minority class, SMOTE synthesizes new cases. To do this, a data point is randomly selected from the minority class and its $K$-nearest neighbors ($K$NNs) are determined. Chawla et al. (2002) used five neighbors in their analyses, but different values can be used depending on the data. The new synthetic data point is a random combination of the predictors of the randomly selected data point and its neighbors. While the SMOTE algorithm adds new samples to the minority class via up-sampling, it also can down-sample cases from the majority class via random sampling in order to help balance the training set.

Figure 16.3 shows an example of sampling methods using a simulated data set. The original data contain 168 samples from the first class and 32 from the second (a 5.2:1 ratio). The down-sampled version of the data reduced the total sample size to 64 cases evenly split between the classes. The up-sampled data have 336 cases, now with 168 events. The SMOTE version of the data has a smaller imbalance (with a 1.3:1 ratio) resulting from having 128 samples from the first class and 96 from the second. For the most part,

the new synthetic samples are very similar to other cases in the data set; the space encompassing the minority class is being filled in instead of expanding.

It should be noted that when using modified versions of the training set, resampled estimates of model performance can become biased. For example, if the data are up-sampled, resampling procedures are likely to have the same sample in the cases that are used to build the model as well as the holdout set, leading to optimistic results. Despite this, resampling methods can still be effective at tuning the models.

A substantial amount of research has been conducted on the effectiveness of using sampling procedures to combat skewed class distributions, most notably Weiss and Provost (2001b), Batista et al. (2004), Van Hulse et al. (2007), Burez and Van den Poel (2009), and Jeatrakul et al. (2010). These and other publications show that, in many cases, sampling can mitigate the issues caused by an imbalance, but there is no clear winner among the various approaches. Also, many modeling techniques react differently to sampling, further complicating the idea of a simple guideline for which procedure to use.

These sampling methods were applied to the random forest models for the insurance data using the same tuning process as the original model. The ROC and lift curves for the evaluation set for three of the random forest models are shown in Fig. 16.4. Numerical summaries of the models are contained in Table 16.3. In this table, the area under the ROC curve is calculated for the evaluation and test sets. Using the evaluation set, new cutoffs for each model were derived by choosing the point on the ROC curve closest to the optimal model. The sensitivity and specificity values in this table are the result of applying those cutoffs to the test set.

The results show that the up-sampling procedure had no real improvement on the area under the curve. SMOTE showed an improvement in the evaluation set, but the increase in the area under the ROC curve was not reproduced in the larger test set. Simple down-sampling of the data also had a limited effect on model performance. However, down-sampling inside the random forest model had robust areas under the ROC curve in both data sets. This may be due to using independent realizations of the majority class in each tree. In all, the results are mixed. While these improvements are modest, the sampling approaches have the benefit of enabling better trade-offs between sensitivity and specificity (unlike modifying cutoffs or prior probabilities).

## 16.8 Cost-Sensitive Training

Instead of optimizing the typical performance measure, such as accuracy or impurity, some models can alternatively optimize a cost or loss function that differentially weights specific types of errors. For example, it may be

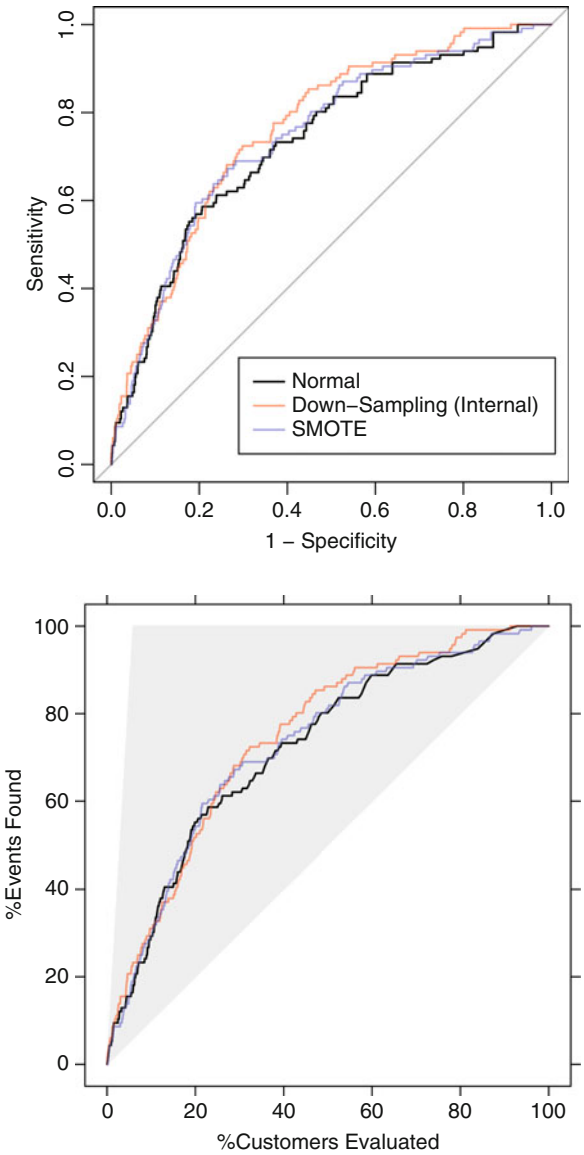



Fig. 16.4: *Top*: Test set ROC curves for three of the random forest models. *Bottom*: The corresponding lift plots

appropriate to believe that misclassifying true events (false negatives) is $X$ times as costly as incorrectly predicting nonevents (false positives). Incorporation of specific costs during model training may bias the model towards less

Table 16.3: A summary of results for random forest models with different sampling mechanisms

| Method | Evaluation | Test | | |
|---|---|---|---|---|
| | ROC | ROC | Sensitivity | Specificity |
| Original | 0.757 | 0.738 | 64.4 | 75.9 |
| Down-sampling | 0.794 | 0.730 | 81.4 | 70.3 |
| Down-sampling (Internal) | 0.792 | 0.764 | 78.0 | 68.3 |
| Up-sampling | 0.755 | 0.739 | 71.2 | 68.1 |
| SMOTE | 0.767 | 0.747 | 78.0 | 67.7 |

The test set sensitivity and specificity values were determined using the optimal cutoff derived from the evaluation set ROC curve

frequent classes. Unlike using alternative cutoffs, unequal costs can affect the model parameters and thus have the potential to make true improvements to the classifier.

For support vector machine (SVM) models, costs can be associated with specific classes (as opposed to specific types of errors). Recall that these models control the complexity using a cost function that increases the penalty if samples are on the incorrect side of the current class boundary. For class imbalances, unequal costs for each class can adjust the parameters to increase or decrease the sensitivity of the model to particular classes (Veropoulos et al. 1999). Note that this approach is different from one where specific types of errors can have differential costs. For support vector machines (SVMs), the entire class can be given increased importance. For two classes, these two approaches are similar.

One consequence of this approach is that class probabilities cannot be generated for the model, at least in the available implementation. Therefore we cannot calculate an ROC curve and must use a different performance metric. Instead we will now use the Kappa statistic, sensitivity, and specificity to evaluate the impact of weighted classes.

For the SVM model, we analyzed both weighted and unweighted models and compared the performance results. When tuning the models, the unweighted SVM required a fairly large value of the SVM cost parameter (256) to optimize the Kappa statistic. Applying the unweighted model to the test set, the Kappa statistic was 0.121 with a corresponding sensitivity of 15.5 % and a specificity of 95.7 %. The effects of weights ranging between 6 and 25 were assessed for the training set (using resampling) and the evaluation set. Across the range of weights selected, the model performance based on the Kappa statistic, sensitivity, and specificity was very similar for the two data sets (Fig. 16.5). Smaller weights optimize Kappa, while moderate to

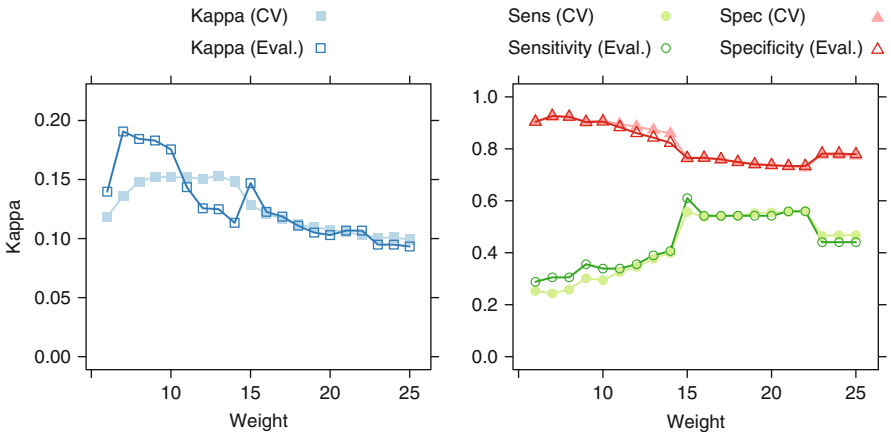



Fig. 16.5: Tuning results for class weighted support vector machine determined with cross-validation and the evaluation set

high weights optimize sensitivity. Using the plot, the modeler can decide the appropriate operating characteristics of the model for the problem at hand as described in Sect. 16.3.

Additionally, many classification tree models can incorporate differential costs, including CART and C5.0 trees. The potential cost of a prediction takes into account several factors (Johnson and Wichern 2001):

- The cost of the particular mistake
- The probability of making that mistake
- The prior probability of the classes

To illustrate how costs can be taken into account, let's return to the concepts and notation used in Sect. 13.6. Let $\pi_i$ represent the prior probability of a sample being in class $i$, and let $Pr[j|i]$ be the probability of mistakenly predicting a class $i$ sample as class $j$. For a two-class problem, the total probability of misclassifying a sample is then

$$Pr[2|1]\pi_1 + Pr[1|2]\pi_2$$

In practice we rarely know $Pr[j|i]$ and instead use the estimated probability of the $i^{th}$ class, $p_i$.

For two classes, the above equation can then be transformed into a rule for classifying a sample into class 1, if

$$\frac{p_1}{p_2} > \frac{\pi_2}{\pi_1}$$

This rule assumes that the costs of misclassifying a class 1 sample into class 2 or a class 2 sample into class 1 are equal. Conveniently, the equation can easily be extended to consider the scenario where the costs are not the same.

Let $C(j|i)$ be the cost of mistakenly predicting a class $i$ sample as class $j$. Then the total expected cost of misclassification is

$$\text{Expected Cost} = C(2|1)Pr[2|1]\pi_1 + C(1|2)Pr[1|2]\pi_2$$

And the revised decision boundary for classifying a sample into class 1 is

$$\frac{p_1}{p_2} > \left(\frac{C(1|2)}{C(2|1)}\right)\left(\frac{\pi_2}{\pi_1}\right)$$

For equal priors, the first class is only predicted when the probability ratio is greater than the cost ratio. If $p_1 = 0.75$ and $p_2 = 0.25$, the second class would be predicted when the cost of erroneously predicting that class is greater than 3.0.

Once the cost of each type of error is specified, CART trees can incorporate them into the training process. Breiman et al. (1984) discuss the use of a generalized Gini criterion:

$$\begin{aligned} Gini^* &= C(1|2)p_1(1-p_1) + C(2|1)p_2(1-p_2) \\ &= [C(1|2) + C(2|1)]\, p_1 p_2 \end{aligned}$$

where $p_1$ and $p_2$ are the observed class probabilities induced by the split. Note that, in this case, the two costs are lumped together when using the Gini index to determine splits. With more than two classes, this same issue arises. As such, using costs with the Gini matrix symmetrizes the costs, that is to say, that the costs $C(i|j)$ and $C(j|i)$ are averaged when determining the overall costs for the split. Breiman et al. (1984) also point out that, in some cases, using unequal costs becomes equivalent to using modified prior probabilities. In their examples, they note that the cost-sensitive trees tended to be smaller than the trees produced by the nominal procedure. The reasoning was that in the growing phase, the tree greedily splits to produce accurate predictions of the potentially costly errors but does not work as hard to be accurate on the remaining errors that have lower costs.

For trees (and rules), the predicted class probabilities (or confidence values) might not be consistent with the discrete class predictions when unequal costs are used. The final class prediction for a sample is a function of the class probability *and* the cost structure. As previously shown, the class probabilities in the terminal node may be appreciably favoring the one class but also have a large expected cost. For this reason, there is a disconnect between the confidence values and the predicted class. Therefore, simple class probabilities (or confidence values) should not be used under these circumstances.

To demonstrate cost-sensitive model training, single CART trees were fit to the data with equal costs as well as a wide range of cost values (2.5 to 30) similar to the previous weighted SVM model. Figure 16.6 shows the results for the training and evaluation sets. The trends shown in these plots are noisy (due to the instability of CART trees), especially for sensitivity, which has

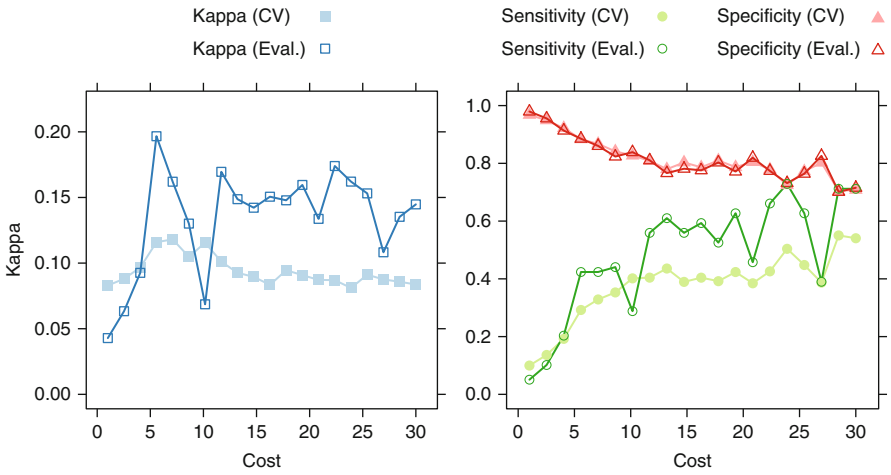



Fig. 16.6: Tuning results for cost-sensitive classification tree models determined with cross-validation and the evaluation set

the smaller sample size. The resampled estimate of the sensitivity was more pessimistic than the evaluation set. The results show that the sensitivity and specificity of the model start to converge at high cost values.

Recall that C5.0 does not use the Gini criterion for splitting. With costs, the previous formula is used and assumes equal prior probabilities[4]:

$$\text{Expected Cost} = C(2|1)p_1 + C(1|2)p_2$$

Again, new samples are predicted as the class associated with the lowest expected cost.

For C5.0, the models were tuned over the model type (i.e., trees or rules) and the number of boosting iterations. Figure 16.7 shows the patterns for cost-sensitive learning with false-negative costs ranging from 5 to 50. Like the SVM analysis, the curves are consistent between the training and evaluation sets. To optimize the concordance via the Kappa statistic, smaller costs values are required and to increase the sensitivity, moderate to large costs are needed. Here, there is a clear trade-off between sensitivity and specificity, but the sensitivity appears to plateau for costs values greater than 25 or 30.

[4] An alternative view of this criterion is that the priors are to be included into the appropriate cost value.

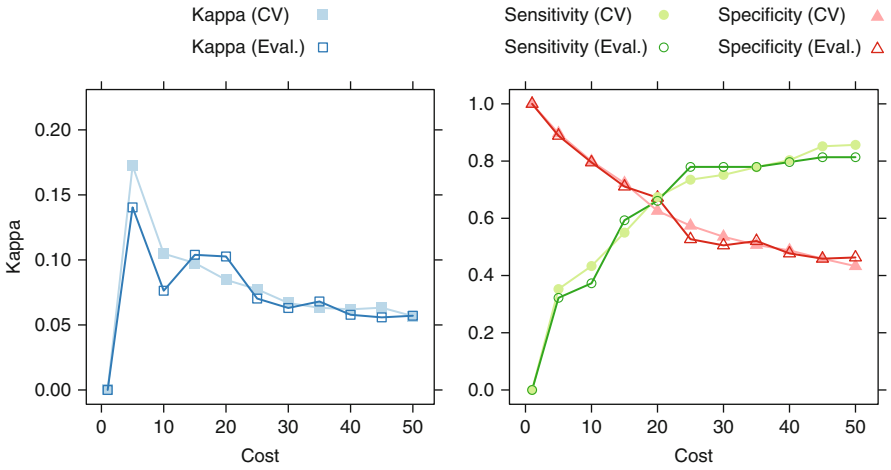



Fig. 16.7: Performance profiles for C5.0 models using cost-sensitive learning

## 16.9 Computing

This section uses the following packages: caret, C50, DMwR, DWD, kernlab, pROC, and rpart.

The insurance data are contained in the DWD package, and can be loaded as follows:

```
> library(DWD)
> data(ticdata)
```

There are several factor variables in the data set. Many of the factor levels have nonstandard characters, such as "%," commas, and other values. When these are converted to dummy variable columns, the values violate the rules for naming new variables. To bypass this issue, we re-encode the names to be more simplistic:

```
> recodeLevels <- function(x)
+   {
+     x <- as.numeric(x)
+     ## Add zeros to the text version:
+     x <- gsub(" ", "0",format(as.numeric(x)))
+     factor(x)
+   }
> ## Find which columns are regular factors or ordered factors
> isOrdered <- unlist(lapply(ticdata, is.ordered))
> isFactor <- unlist(lapply(ticdata, is.factor))
> convertCols <- names(isOrdered)[isOrdered | isFactor]


> for(i in convertCols) ticdata[,i] <- recodeLevels(ticdata[,i])
> ## Make the level 'insurance' the first factor level
```

```
> ticdata$CARAVAN <- factor(as.character(ticdata$CARAVAN),
+                           levels = rev(levels(ticdata$CARAVAN)))
```

The training and test sets were created using stratified random sampling:

```
> library(caret)
> ## First, split the training set off
> set.seed(156)
> split1 <- createDataPartition(ticdata$CARAVAN, p = .7)[[1]]
> other     <- ticdata[-split1,]
> training  <- ticdata[ split1,]


> ## Now create the evaluation and test sets
> set.seed(934)
> split2 <- createDataPartition(other$CARAVAN, p = 1/3)[[1]]
> evaluation  <- other[ split2,]
> testing     <- other[-split2,]
> ## Determine the predictor names
> predictors <- names(training)[names(training) != "CARAVAN"]
```

Dummy variables are useful for several models being fit in this section. The `randomForest` function has a limitation that all factor predictors must not have more than 32 levels. The customer type predictor has 39 levels, so a predictor set of dummy variables is created for this and other models using the `model.matrix` function:

```
> ## The first column is the intercept, which is eliminated:
> trainingInd   <- data.frame(model.matrix(CARAVAN ~ .,
+                                          data = training))[,-1]
> evaluationInd <- data.frame(model.matrix(CARAVAN ~ .,
+                                          data = evaluation))[,-1]
> testingInd    <- data.frame(model.matrix(CARAVAN ~ .,
+                                          data = testing))[,-1]


> ## Add the outcome back into the data set
> trainingInd$CARAVAN   <- training$CARAVAN
> evaluationInd$CARAVAN <- evaluation$CARAVAN
> testingInd$CARAVAN    <- testing$CARAVAN


> ## Determine a predictor set without highly sparse and unbalanced
  distributions:
> isNZV <- nearZeroVar(trainingInd)
> noNZVSet <- names(trainingInd)[-isNZV]
```

To obtain different performance measures, two wrapper functions were created:

```
> ## For accuracy, Kappa, the area under the ROC curve,
> ## sensitivity and specificity:
> fiveStats <- function(...) c(twoClassSummary(...),
+                              defaultSummary(...))
> ## Everything but the area under the ROC curve:
> fourStats <- function (data, lev = levels(data$obs), model = NULL)
```

```
+ {
+
+   accKapp <- postResample(data[, "pred"], data[, "obs"])
+   out <- c(accKapp,
+            sensitivity(data[, "pred"], data[, "obs"], lev[1]),
+            specificity(data[, "pred"], data[, "obs"], lev[2]))
+   names(out)[3:4] <- c("Sens", "Spec")
+   out
+ }
```

Two control functions are developed for situations when class probabilities can be created and when they cannot:

```
> ctrl <- trainControl(method = "cv",
+                      classProbs = TRUE,
+                      summaryFunction = fiveStats,
+                      verboseIter = TRUE)
> ctrlNoProb <- ctrl
> ctrlNoProb$summaryFunction <- fourStats
> ctrlNoProb$classProbs <- FALSE
```

The three baseline models were fit with the syntax:

```
> set.seed(1410)
> rfFit <- train(CARAVAN ~ ., data = trainingInd,
+                method = "rf",
+                trControl = ctrl,
+                ntree = 1500,
+                tuneLength = 5,
+                metric = "ROC")


> set.seed(1410)
> lrFit <- train(CARAVAN ~ .,
+                data = trainingInd[, noNZVSet],
+                method = "glm",
+                trControl = ctrl,
+                metric = "ROC")


> set.seed(1401)
> fdaFit <- train(CARAVAN ~ ., data = training,
+                 method = "fda",
+                 tuneGrid = data.frame(.degree = 1, .nprune = 1:25),
+                 metric = "ROC",
+                 trControl = ctrl)
>
```

A data frame is used to house the predictions from different models:

```
> evalResults <- data.frame(CARAVAN = evaluation$CARAVAN)
> evalResults$RF <- predict(rfFit,
+                           newdata = evaluationInd,
+                           type = "prob")[,1]
```

```
> evalResults$FDA <- predict(fdaFit,
+                            newdata = evaluation[, predictors],
+                            type = "prob")[,1]
> evalResults$LogReg <- predict(lrFit,
+                               newdata = valuationInd[, noNZVSet],
+                               type = "prob")[,1]
```

The ROC and lift curves are created from these objects. For example:

```
> library(pROC)
> rfROC <- roc(evalResults$CARAVAN, evalResults$RF,
+              levels = rev(levels(evalResults$CARAVAN)))
> ## Create labels for the models:
> labs <- c(RF = "Random Forest", LogReg = "Logistic Regression",
+           FDA = "FDA (MARS)")
> lift1 <- lift(CARAVAN ~ RF + LogReg + FDA, data = evalResults,
+               labels = labs)

> rfROC
  Call:
  roc.default(response = evalResults$CARAVAN, predictor = evalResults$RF,
       levels = rev(levels(evalResults$CARAVAN)))

  Data: evalResults$RF in 924 controls (evalResults$CARAVAN noinsurance) <
        59 cases (evalResults$CARAVAN insurance).
  Area under the curve: 0.7569
> lift1
  Call:
  lift.formula(x = CARAVAN ~ RF + LogReg + FDA, data = evalResults,
               labels = labs)

  Models: Random Forest, Logistic Regression, FDA (MARS)
  Event: insurance (6%)
```

To plot the curves:

```
> plot(rfROC, legacy.axes = TRUE)
> xyplot(lift1,
+        ylab = "%Events Found", xlab =  "%Customers Evaluated",
+        lwd = 2, type = "l")
```

## *Alternate Cutoffs*

After the ROC curve has been created, there are several functions in the pROC package that can be used to investigate possible cutoffs. The `coords` function returns the points on the ROC curve as well as deriving new cutoffs. The main arguments are `x`, which specifies what should be returned. A value of `x = "all"` will return the coordinates for the curve and their associated cutoffs. A value of `"best"` will derive a new cutoff. Using `x = "best"` in conjunction with the `best.method` (either `"youden"` or `"closest.topleft"`) can be informative:

```
> rfThresh <- coords(rfROC, x = "best", best.method = "closest.topleft")
> rfThresh
    threshold specificity sensitivity
  0.06433333  0.75865801  0.64406780
```

For this, new predicted classes can be calculated:

```
> newValue <- factor(ifelse(evalResults$RF > rfThresh,
+                           "insurance", "noinsurance"),
+                    levels = levels(evalResults$CARAVAN))
```

## *Sampling Methods*

The caret package has two functions, `downSample` and `upSample`, that readjust the class frequencies. Each takes arguments for the predictors (called `x`) and the outcome class (`y`). Both functions return a data frame with the sampled version of the training set:

```
> set.seed(1103)
> upSampledTrain <- upSample(x = training[,predictors],
+                            y = training$CARAVAN,
+                            ## keep the class variable name the same:
+                            yname = "CARAVAN")
> dim(training)
  [1] 6877   86
> dim(upSampledTrain)
  [1] 12932    86
> table(upSampledTrain$CARAVAN)
    insurance noinsurance
         6466        6466
```

The down-sampling function has the same syntax. A function for SMOTE can be found in the DMwR package. It takes a model formula as an input, along with parameters (such as the amount of over- and under-sampling and the number of neighbors). The basic syntax is

```
> library(DMwR)
> set.seed(1103)
> smoteTrain <- SMOTE(CARAVAN ~ ., data = training)
> dim(smoteTrain)
  [1] 2877   86
> table(smoteTrain$CARAVAN)
    insurance noinsurance
         1233        1644
```

These data sets can be used as inputs into the previous modeling code.

### *Cost-Sensitive Training*

Class-weighted SVMs can be created using the kernlab package. The syntax for the `ksvm` function is the same as previous descriptions, but the `class.weights` argument is put to use. The `train` function has similar syntax:

```
> library(kernlab)
> ## We will train over a large cost range, so we precompute the sigma
> ## parameter and make a custom tuning grid:
> set.seed(1157)
> sigma <- sigest(CARAVAN ~ ., data = trainingInd[, noNZVSet], frac = .75)
> names(sigma) <- NULL
> svmGrid <- data.frame(.sigma = sigma[2],
+                       .C = 2^seq(-6, 1, length = 15))
> ## Class probabilities cannot be generated with class weights, so
> ## use the control object 'ctrlNoProb' to avoid estimating the
> ## ROC curve.
> set.seed(1401)
> SVMwts <- train(CARAVAN ~ .,
+                 data = trainingInd[, noNZVSet],
+                 method = "svmRadial",
+                 tuneGrid = svmGrid,
+                 preProc = c("center", "scale"),
+                 class.weights = c(insurance = 18, noinsurance = 1),
+                 metric = "Sens",
+                 trControl = ctrlNoProb)


> SVMwts
  6877 samples
   203 predictors
     2 classes: 'insurance', 'noinsurance'

  Pre-processing: centered, scaled
  Resampling: Cross-Validation (10-fold)

  Summary of sample sizes: 6189, 6190, 6190, 6189, 6189, 6189, ...

  Resampling results across tuning parameters:

    C       Accuracy  Kappa   Sens   Spec
    0.0156  0.557     0.0682  0.742  0.545
    0.0221  0.614     0.0806  0.691  0.609
    0.0312  0.637     0.0864  0.669  0.635
    0.0442  0.644     0.0883  0.662  0.643
    0.0625  0.658     0.0939  0.657  0.658
    0.0884  0.672     0.0958  0.633  0.674
    0.125   0.684     0.101   0.625  0.688
    0.177   0.7       0.106   0.611  0.705
    0.25    0.711     0.108   0.591  0.719
    0.354   0.724     0.111   0.572  0.734
    0.5     0.737     0.112   0.543  0.75
```

```
0.707   0.75      0.109   0.506  0.765
1       0.765     0.104   0.46   0.785
1.41    0.776     0.097   0.416  0.799
2       0.791     0.102   0.394  0.817

Tuning parameter 'sigma' was held constant at a value of 0.00245
Sens was used to select the optimal model using  the largest value.
The final values used for the model were C = 0.0156 and sigma = 0.00245.
```

(The standard deviation columns were not shown to save space) Prediction uses the same syntax as unweighted models.

For cost-sensitive CART models, the rpart package is used with the `parms` argument, which is a list of fitting options. One option, `loss`, can take a matrix of costs:

```
> costMatrix <- matrix(c(0, 1, 20, 0), ncol = 2)
> rownames(costMatrix) <- levels(training$CARAVAN)
> colnames(costMatrix) <- levels(training$CARAVAN)
> costMatrix
             insurance noinsurance
insurance            0          20
noinsurance          1           0
```

Here, there would be a 20-fold higher cost of a false negative than a false positive. To fit the model:

```
> library(rpart)
> set.seed(1401)
> cartCosts <- train(x = training[,predictors],
+                    y = training$CARAVAN,
+                    method = "rpart",
+                    trControl = ctrlNoProb,
+                    metric = "Kappa",
+                    tuneLength = 10,
+                    parms = list(loss = costMatrix))
```

Similar to the support vector machine model, the syntax for generating class predictions is the same as the nominal model. However, any class probabilities generated from this model may not match the predicted classes (which are a function of the cost and the probabilities).

C5.0 has similar syntax to `rpart` by taking a cost matrix, although this function uses the transpose of the cost matrix structure used by `rpart`:

```
> c5Matrix <- matrix(c(0, 20, 1, 0), ncol = 2)
> rownames(c5Matrix) <- levels(training$CARAVAN)
> colnames(c5Matrix) <- levels(training$CARAVAN)
> c5Matrix
             insurance noinsurance
insurance            0           1
noinsurance         20           0
```

```
> library(C50)
> set.seed(1401)
> C5Cost <- train(x = training[, predictors],
+                 y = training$CARAVAN,
+                 method = "C5.0",
+                 metric = "Kappa",
+                 cost = c5Matrix,
+                 trControl = ctrlNoProb)
```

When employing costs, the `predict` function for this model only produces the discrete classes (i.e. no probabilities).

## Exercises

**16.1.** The "adult" data set at the UCI Machine Learning Repository is derived from census records.[5] In these data, the goal is to predict whether a person's income was large (defined in 1994 as more than \$50K) or small. The predictors include educational level, type of job (e.g., never worked, and local government), capital gains/losses, work hours per week, native country, and so on.[6] After filtering out data where the outcome class is unknown, there were 48842 records remaining. The majority of the data were associated with a small income level (75.9 %).

The data are contained in the arules package and the appropriate version can be loaded using `data(AdultUCI)`.

(a) Load the data and investigate the predictors in terms of their distributions and potential correlations.
(b) Determine an appropriate split of the data.
(c) Build several classification models for these data. Do the results favor the small income class?
(d) Is there a good trade-off that can be made between the sensitivity and specificity?
(e) Use sampling methods to improve the model fit.
(f) Do cost-sensitive models help performance?

**16.2.** The direct marketing data of Larose (2006, Chap. 7) discussed previously in Chap. 11 can be found at the author's web site.[7] The goal of the

[5] These data are first referenced in Kohavi (1996). A description of the data collection and a summary of the results of previous models can be found at the UCI Machine Learning Repository (http://archive.ics.uci.edu/ml/machine-learning-databases/adult/adult.names).

[6] Another attribute, "fnlwgt," only contains information about the data collection process and should not be used as a predictor.

[7] http://www.dataminingconsultant.com/data.

analysis was to predict which customers would respond to a promotional opportunity via mail.

Of the 65220 customers in the data set, 16.6 % responded to the promotion. The predictors in the data set include

- Spending habits, in aggregate and broken down by month and frequently visited store locations
- The types of products purchased
- Time between visits
- Historical promotion response rates
- Predefined customer cluster memberships

Larose (2006) discussed the class imbalance for this problem and demonstrates several techniques to achieve effective results.

(a) Read the data into R, conduct exploratory analyses, and determine the best method for encoding the predictors.
(b) Determine an appropriate split of the data and build several classification models for these data.
(c) Construct lift plots to understand a possible strategy regarding how many customers should be contacted to capture an estimated 60 % of responders.
(d) Use sampling methods with several models. Do these models have better lift charts, and can they be used to contact fewer customers to achieve a 60 % response rate (of those who were given the promotion)?